



**Department of Electrical and Computer Engineering
North South University**

Directed Research

**Visual Similarity Attacks on QR Codes Using Machine
Learning-Based UI Deception Detection**

MINHAJUL AZAM

2222035042

**Faculty Advisor:
Muhammad Shafayat Oshman
Lecturer & Researcher
ECE Department**

Spring, 2026

APPROVAL

Minhajul Azam (2222035042) from Electrical and Computer Engineering Department of North South University, have worked on the Directed Research Project titled “**Visual Similarity Attacks on QR Codes Using Machine Learning-Based UI Deception Detection**” under the supervision of Muhammad Shafayat Oshman an partial fulfillment of the requirement for the degree of Bachelors of Science in Engineering and has been accepted as satisfactory.

Supervisor’s Signature

.....

Muhammad Shafayat Oshman

Lecturer & Researcher

Department of Electrical and Computer Engineering

North South University

Dhaka, Bangladesh.

Chairman’s Signature

.....

Dr. Mohammad Abdul Matin

Professor & Chairman

Department of Electrical and Computer Engineering

North South University

Dhaka, Bangladesh.

DECLARATION

This is to declare that this project/directed research is my original work. No part of this work has been submitted elsewhere partially or fully for the award of any other degree or diploma. All project related information will remain confidential and shall not be disclosed without the formal consent of the project supervisor. Relevant previous works presented in this report have been properly acknowledged and cited. The plagiarism policy, as stated by the supervisor, has been maintained.

Student name & Signature

Student Name

ACKNOWLEDGEMENTS

The authors would like to express their heartfelt gratitude towards their project and research supervisor, **Muhammad Shafayat Oshman** Lecturer & Researcher, Department of Electrical and Computer Engineering, North South University, Bangladesh, for his invaluable support, precise guidance and advice pertaining to the experiments, research and theoretical studies carried out during the course of the current project and also in the preparation of the current report.

Furthermore, the authors would like to thank the Department of Electrical and Computer Engineering, North South University, Bangladesh for facilitating the research. We would also like to thank my friends X, Y, and Z for helping us in this project. The authors would also like to thank their loved ones for their countless sacrifices and continual support.

ABSTRACT

Visual Similarity Attacks on QR Codes Using Machine Learning-Based UI Deception Detection

Quick Response (QR) codes have become a common part of everyday digital life and are widely used in online payments, advertisements, authentication systems, ticketing platforms, and mobile applications because of their speed, convenience, and ease of use. However, the growing use of QR codes has also increased cybersecurity risks. One of the most concerning threats is QR phishing, also known as “quishing”, where attackers create or manipulate QR codes to redirect users to fake websites, fraudulent login pages, or malicious applications. Since users cannot visually understand the actual content hidden inside a QR code before scanning it, attackers can easily take advantage of user trust through deceptive and visually convincing QR-based attacks.

Most existing research mainly focuses on analyzing URLs after the QR code has already been scanned and decoded. Although these methods can detect suspicious links, they often fail to address visual similarity attacks, fake branding, UI-based deception techniques, and other trust manipulation strategies commonly used in modern phishing attacks. In addition, many existing approaches rely on traditional machine learning methods instead of advanced deep learning-based image analysis.

This research presents a deep learning-based framework for detecting malicious QR codes using image-based analysis. Several deep learning architectures, including Convolutional Neural Network (CNN), MobileNetV2, ResNet50, EfficientNetB0, DenseNet121, VGG16, and InceptionV3, were trained and evaluated using a large dataset containing both legitimate and phishing QR codes. The models were evaluated using performance metrics such as accuracy, precision, recall, F1-score, confusion matrix analysis, and training-validation performance graphs.

The experimental results show that EfficientNetB0, MobileNetV2, and the custom CNN model achieved the highest performance with approximately 99% accuracy, while InceptionV3 achieved around 89% accuracy. The findings suggest that deep learning-based QR image analysis

can significantly improve malicious QR code detection and support the development of more secure and intelligent QR-based cybersecurity systems. This research may also contribute to future work related to visual deception analysis, real-time QR security applications, and advanced phishing prevention systems.

TABLE OF CONTENTS

LETTER OF TRANSMITTAL	Error! Bookmark not defined.
APPROVAL	ii
DECLARATION	iii
ACKNOWLEDGEMENTS.....	iv
ABSTRACT.....	v
TABLE OF CONTENTS.....	vii
LIST OF FIGURES	ix
LIST OF TABLES.....	x
Chapter 1 Introduction	1
1.1 Background and Motivation.....	1
1.2 Purpose and Goal of the Project.....	1
1.3 Organization of the Report.....	1
Chapter 2 Research Literature Review	3
2.1 Existing Research and Limitations.....	3
Chapter 3 Methodology	4
3.1 System Design.....	4
3.2 Hardware and/or Software Components	4
3.3 Hardware and/or Software Implementation	7
3.4 Challenges and Accuracy Improvement Process	8
Chapter 4 Investigation/Experiment, Result, Analysis and Discussion.....	10
4.1 Experimental Setup	10
4.2 Model Performance Comparison	12
4.3 Confusion Matrix Analysis	13

4.4 Training and Validation Accuracy	14
4.5 Training and Validation Loss	15
Chapter 5 Conclusions	16
5.1 Summary	16
5.2 Limitations	16
5.3 Future Improvement.....	16
References.....	17

LIST OF FIGURES

1. Figure 3.1: Flowchart of QR Phishing Detection Framework.....	4
2. Figure 4.1: Accuracy Comparison of Deep Learning Models.....	11
3. Figure 4.2 Precision, Recall and F1-Score Analysis.....	12
4. Figure 4.3 Confusion Matrix Analysis.....	13
5. Figure 4.4 Training and Validation Accuracy.....	14
6. Figure 4.5: Training and Validation Loss.....	15

LIST OF TABLES

Table 3.1: Components and Purpose of the Research System 6

Table 3.2: Software and Hardware Tools Used in the Research 7

Chapter 1 Introduction

1.1 Background and Motivation

Quick Response (QR) codes are widely used in online payment systems, advertisements, authentication platforms, ticketing services, and mobile applications because of their speed and convenience. With the increasing use of smartphones and digital transactions, QR codes have become a common part of everyday digital communication.

However, the growing use of QR codes has also increased cybersecurity threats. One major threat is QR phishing, also known as “quishing”, where attackers use malicious QR codes to redirect users to fake websites or harmful applications. Since users cannot visually verify the hidden content inside a QR code before scanning it, attackers can easily exploit user trust through deceptive QR-based attacks.

Most existing research focuses mainly on analyzing URLs after the QR code has already been scanned. These approaches often fail to address visual deception techniques and image-based QR attacks. Therefore, this research focuses on developing a deep learning-based QR phishing detection system using QR image analysis.

1.2 Purpose and Goal of the Project

The main purpose of this research is to develop a deep learning-based framework for detecting malicious QR codes using image-based analysis techniques. This research aims to analyze QR code images directly instead of depending only on decoded URL analysis.

Several deep learning models including CNN, MobileNetV2, ResNet50, EfficientNetB0, DenseNet121, VGG16, and InceptionV3 were trained and evaluated using legitimate and phishing QR code datasets. The performance of the models was analyzed using accuracy, precision, recall, F1-score, and confusion matrix analysis.

The goal of this project is to identify the most effective deep learning model for QR phishing detection and contribute to future QR security research.

1.3 Organization of the Report

- Chapter 1 introduces the background, motivation, objectives, and structure of the research.
- Chapter 2 presents the literature review and discusses existing research related to QR phishing attacks and detection methods.
- Chapter 3 describes the proposed methodology, dataset preparation, deep learning architectures, and experimental setup.

- Chapter 4 presents the experimental results, model comparison, and performance analysis.
- Chapter 5 concludes the research and discusses future improvements and research directions.

Chapter 2 Research Literature Review

2.1 Existing Research and Limitations

Several researchers have worked on QR phishing detection and QR security using machine learning and deep learning techniques. In the study “*QRishing: The Susceptibility of Smartphone Users to QR Code Phishing Attacks*,” the authors analyzed user behavior toward malicious QR codes but did not propose any technical detection system. Another study, “*QR Code Security: Mitigating the Issue of Quishing*,” used machine learning algorithms such as Naive Bayes and Logistic Regression to detect phishing URLs extracted from QR codes. Some recent studies also applied deep learning models such as MobileNetV2 for QR image analysis and malicious QR detection.

However, most existing research mainly depends on URL analysis after scanning the QR code and ignores visual similarity attacks, fake branding, and UI-based deception techniques used in modern phishing attacks. Many studies also rely on traditional machine learning methods instead of advanced deep learning-based image analysis. These limitations motivated this research to develop a deep learning-based QR phishing detection framework using image-based QR code analysis and multiple deep learning models including CNN, MobileNetV2, ResNet50, EfficientNetB0, DenseNet121, VGG16, and InceptionV3.

Chapter 3 Methodology

3.1 System Design

Flowchart of QR Phishing Detection Framework

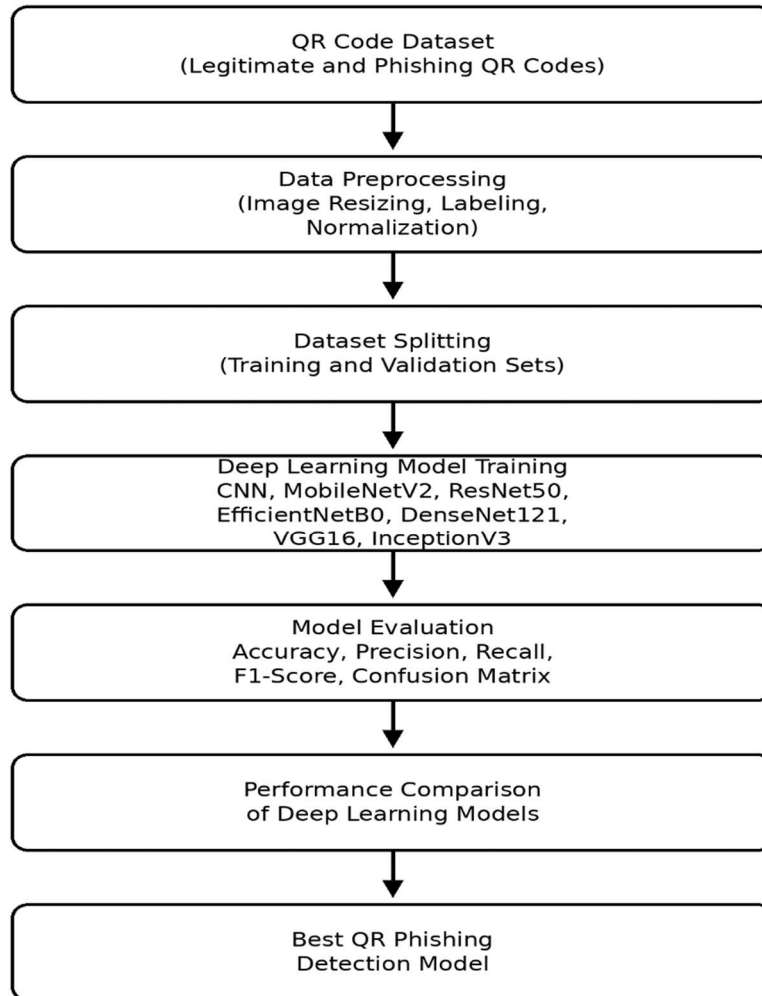


Figure 3.1: Flowchart of QR Phishing Detection Framework

3.2 Hardware and/or Software Components

This research was implemented using both software and hardware resources for QR code dataset generation, preprocessing, model training, evaluation, and visualization. A dataset containing legitimate and phishing QR code images was prepared and used for training and testing purposes. Different preprocessing techniques such as image resizing, normalization, labeling, and dataset splitting were applied before model training. Several deep learning architectures including CNN, MobileNetV2, ResNet50, EfficientNetB0, DenseNet121, VGG16, and InceptionV3 were implemented and evaluated to identify the most effective model for malicious QR code detection.

Python was used as the primary programming language because of its flexibility and strong support for artificial intelligence and deep learning libraries. TensorFlow and Keras were used for implementing and training deep learning models, while OpenCV was used for image preprocessing and QR image handling. NumPy and Pandas were used for data processing and dataset management, and Matplotlib was used for graphical visualization and performance

analysis. The project was developed and tested using Visual Studio Code (VS Code) on a Windows-based computer system.

Component	Purpose
QR Code Dataset	Training and evaluation of models
Image Preprocessing	Resizing, normalization, and labeling
CNN, MobileNetV2, ResNet50, EfficientNetB0, DenseNet121, VGG16, InceptionV3	Deep learning-based QR phishing detection
Python	Main programming language
TensorFlow & Keras	Deep learning model implementation
OpenCV	Image preprocessing and QR image handling
NumPy & Pandas	Dataset processing and data management
Matplotlib	Performance visualization and graph generation
VS Code	Development environment
Windows-based PC	Model training and experimentation

Table 3.1: Components and Purpose of the Research System

TABLE I. A SAMPLE SOFTWARE/HARDWARE TOOLS TABLE

Tool	Functions	Other similar Tools (if any)	Why selected this tool
Python	Main programming language for model development	Java, C++	Easy to use and support AI libraries

Tensorflow	Deep learning model training	Pytorch Lighting	Strong deep learning framework with good documentation
Keras	Building and training neural networks	PyTorch Lightning	Simple and beginner-friendly API
OpenCV	Image preprocessing and QR image handling	PIL, Scikit-image	Efficient image processing support
NumPy	Numerical computation and array operations	MATLAB	Fast mathematical computation
Pandas	Dataset management and analysis	Excel	Easy handling of large datasets
Matplotlib	Graph and visualization generation	Seaborn	Simple plotting and visualization
VS Code	Code development environment	PyCharm	Lightweight and easy to use
Windows PC	Model training and testing	Linux System	Available experimental environment

Table 3.2: Software and Hardware Tools Used in the Research

3.3 Hardware and/or Software Implementation

The implementation of this research project was completed using a software-based deep learning framework for detecting phishing QR codes through image analysis. At first, a large QR code dataset containing both legitimate and phishing QR codes was collected and generated. The dataset was then preprocessed using image resizing, normalization, labeling, and dataset splitting techniques to prepare the data for model training and evaluation.

Several deep learning architectures including CNN, MobileNetV2, ResNet50, EfficientNetB0, DenseNet121, VGG16, and InceptionV3 were implemented using TensorFlow and Keras. The models were trained and evaluated on the prepared dataset using different hyperparameter settings such as batch size, epochs, and preprocessing techniques. During implementation, training accuracy, validation accuracy, training loss, and validation loss were monitored to analyze model performance and reduce overfitting issues.

The system also included performance evaluation modules for generating confusion matrices, accuracy comparison graphs, precision, recall, and F1-score analysis. Among all the implemented models, EfficientNetB0, MobileNetV2, and the custom CNN model achieved the highest performance in detecting malicious QR codes. The complete implementation and experimentation were performed using Python in the Visual Studio Code (VS Code) development environment on a Windows-based computer system.

3.4 Challenges and Accuracy Improvement Process

During the implementation and training process, several challenges were faced while improving the performance of the deep learning models. One of the major challenges was achieving high accuracy while maintaining reasonable training time on a CPU-based Windows system. Some models such as ResNet50, VGG16, DenseNet121, and InceptionV3 required a large amount of training time because of their complex architectures and the large QR image dataset.

Another challenge was the variation in model performance. Certain models initially produced low accuracy and high validation loss due to inappropriate preprocessing methods, insufficient hyperparameter tuning, and overfitting issues. To improve the performance, several techniques were applied including image preprocessing, normalization, data augmentation, learning rate adjustment, dropout layers, batch size optimization, and transfer learning with pretrained ImageNet weights.

Different preprocessing techniques were also tested for different architectures because some models required specific preprocessing functions instead of simple image rescaling. The batch size

and number of epochs were adjusted multiple times to balance training speed and model performance. In addition, some deep learning models such as EfficientNetB0 and MobileNetV2 performed significantly better because of their efficient feature extraction capability and optimized architecture design.

Although GPU acceleration was not available in the experimental environment, several optimization strategies were used to reduce training time and improve model accuracy. The final experimental results showed that EfficientNetB0, MobileNetV2, and the custom CNN model achieved the highest performance with approximately 99% accuracy.

Chapter 4 Investigation/Experiment, Result, Analysis and Discussion

This chapter presents the experimental results and performance analysis of the proposed QR phishing detection system. The experiments were conducted using a dataset containing legitimate and phishing QR code images. The dataset was divided into training and validation sets, and different deep learning models were trained and evaluated to compare their performance in detecting malicious QR codes.

Seven deep learning models were implemented and tested in this research: CNN, MobileNetV2, ResNet50, EfficientNetB0, DenseNet121, VGG16, and InceptionV3. The models were evaluated using accuracy, precision, recall, F1-score, confusion matrix, training accuracy, validation accuracy, training loss, and validation loss.

4.1 Experimental Setup

The experiments were performed using Python, TensorFlow, and Keras in the VS Code environment. All QR code images were resized and preprocessed before model training. The models were trained using different preprocessing techniques depending on the architecture. Transfer learning models such as MobileNetV2, EfficientNetB0, ResNet50, DenseNet121, VGG16, and InceptionV3 used pretrained ImageNet weights.

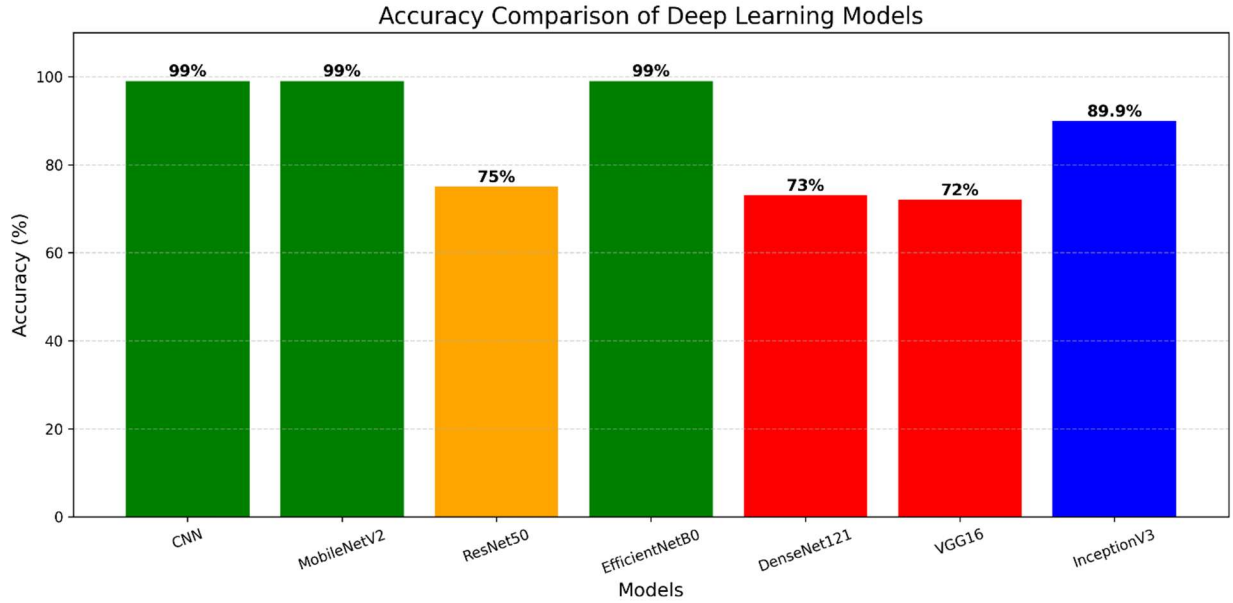


Figure 4.1: Accuracy Comparison of Deep Learning Models

Figure 4.1 presents the accuracy comparison of different deep learning models used in this research for QR phishing detection. The experimental results demonstrate that EfficientNetB0, MobileNetV2, and the custom CNN model achieved the best overall performance with approximately 99% accuracy. These models were able to classify legitimate and phishing QR codes with very high reliability and fewer misclassifications. InceptionV3 also produced satisfactory performance with around 89% accuracy after additional optimization and fine-tuning.

In contrast, ResNet50, DenseNet121, and VGG16 achieved comparatively lower accuracy levels ranging between 72% and 75%. The lower performance of these models may be associated with higher architectural complexity, preprocessing sensitivity, and longer training requirements on a CPU-based system. The overall findings indicate that EfficientNetB0 and MobileNetV2 are more suitable for QR phishing detection because of their efficient feature extraction capability, optimized architecture, and better generalization performance for image-based QR code analysis.

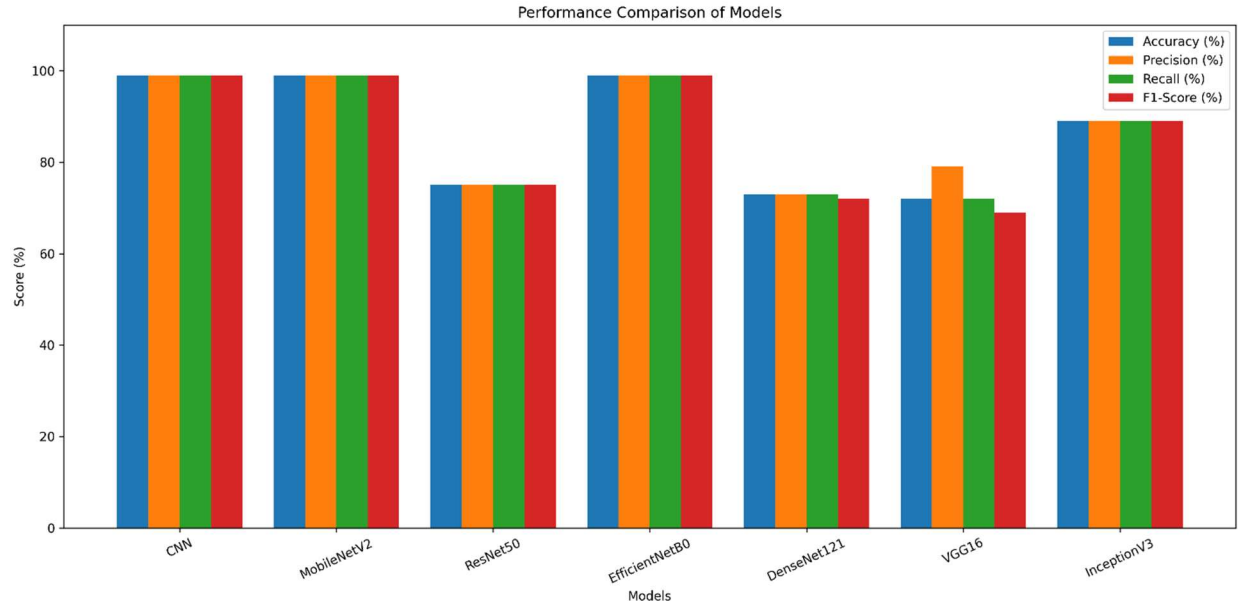


Figure 4.2: Precision, Recall and F1-Score Analysis

Figure 4.2 presents the comparison of accuracy, precision, recall, and F1-score among all the trained deep learning models used in this research. The graph shows that EfficientNetB0, the custom CNN model, and MobileNetV2 achieved the best overall performance across all evaluation metrics. These models demonstrated strong classification capability by accurately identifying both legitimate and phishing QR codes with high consistency and fewer false predictions.

InceptionV3 also achieved comparatively good performance after model optimization and fine-tuning, while ResNet50, DenseNet121, and VGG16 showed lower scores in several evaluation metrics. The comparatively lower performance of these models may be related to architectural complexity, preprocessing sensitivity, and longer training requirements on limited hardware resources. Overall, the results highlight the effectiveness of deep learning-based QR image analysis for phishing detection and indicate that EfficientNetB0 and MobileNetV2 are more reliable and efficient for real-world QR security applications.

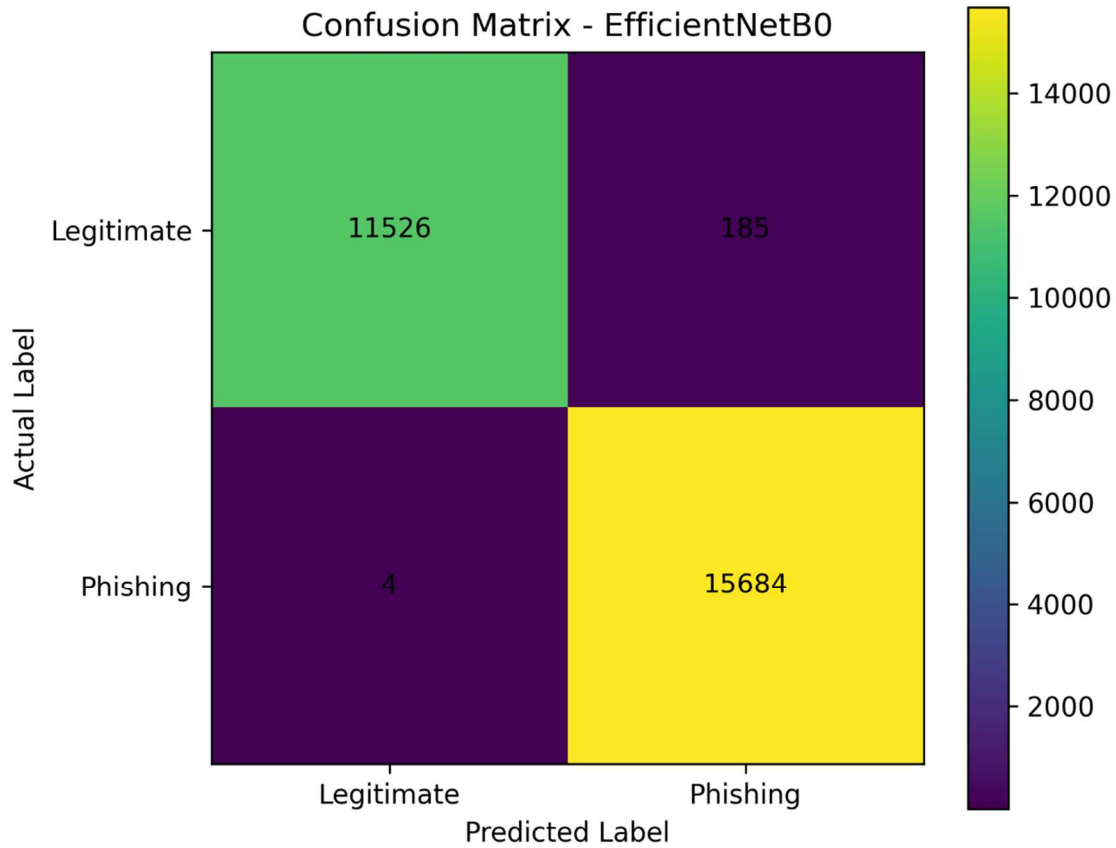


Figure 4.3: Confusion Matrix Analysis

Figure 4.3 shows the confusion matrix of the EfficientNetB0 model used in this research for QR phishing detection. The confusion matrix demonstrates that the model correctly classified the majority of both legitimate and phishing QR codes with only a very small number of misclassifications. Most phishing QR codes were accurately detected, while only a few legitimate QR codes were incorrectly classified as phishing.

The low number of false positives and false negatives indicates the strong classification capability and robustness of the proposed detection system. This result highlights the effectiveness of EfficientNetB0 in extracting important visual features from QR code images and distinguishing between legitimate and malicious patterns. The overall performance of the model confirms that EfficientNetB0 is highly reliable for image-based QR phishing detection and can contribute to the development of more secure QR-based cybersecurity systems.

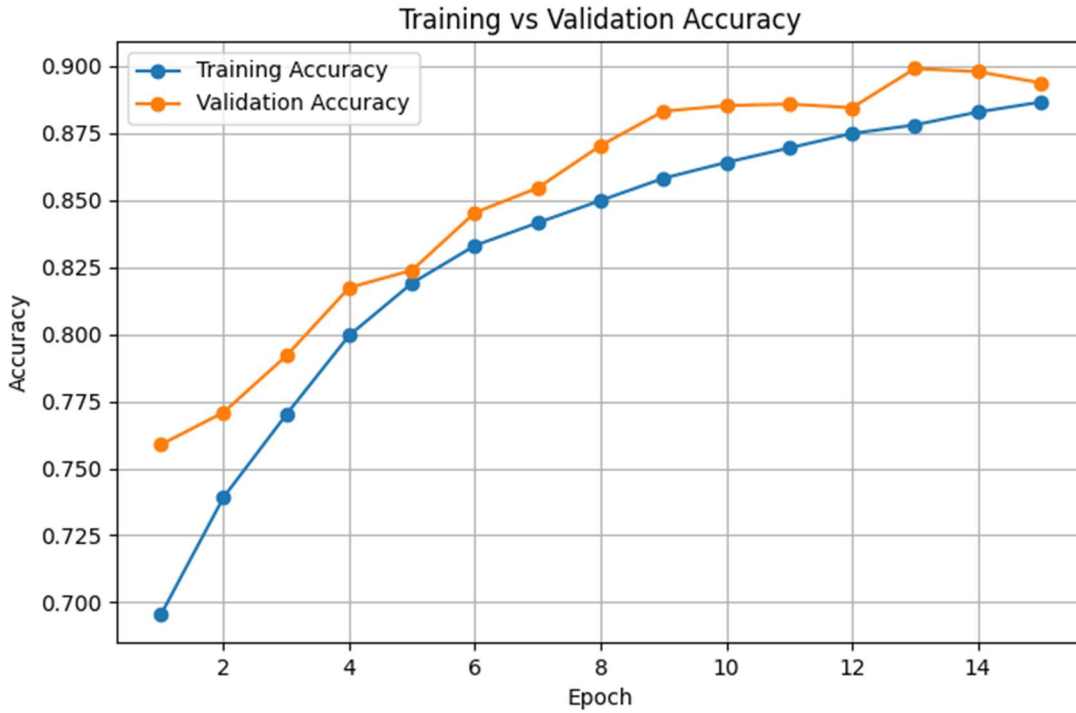


Figure 4.4: Training and Validation Accuracy

Figure 4.4 illustrates the training and validation accuracy of the proposed deep learning model over multiple training epochs. Both training and validation accuracy gradually increased throughout the training process, indicating that the model was successfully learning important features from the QR code images. The steady improvement in accuracy demonstrates effective learning behavior and proper optimization during training.

The validation accuracy remained relatively close to the training accuracy across different epochs, which suggests reduced overfitting and stable model generalization performance. This indicates that the model was able to perform well not only on the training dataset but also on unseen validation data. The smooth trend of the accuracy curves further confirms the reliability and effectiveness of the proposed QR phishing detection framework.



Figure 4.5: Training and Validation Loss

Figure 4.5 presents the training and validation loss curves observed during the training process of the proposed deep learning model. The loss values gradually decreased as the number of training epochs increased, which indicates that the model was successfully learning from the QR code dataset and reducing classification errors over time. The continuous decrease in loss demonstrates effective optimization and proper adjustment of the model parameters during training.

The training and validation loss curves also showed smooth and stable convergence without significant fluctuations, indicating efficient and reliable training performance. In addition, the validation loss remained close to the training loss throughout most of the epochs, suggesting reduced overfitting and improved generalization capability of the model. These results confirm that the proposed deep learning framework achieved stable learning behavior and effective performance for QR phishing detection.

Chapter 5 Conclusions

5.1 Summary

This research explored how deep learning can be used to detect phishing QR codes through visual analysis. Several deep learning models, including CNN, MobileNetV2, ResNet50, EfficientNetB0, DenseNet121, VGG16, and InceptionV3, were trained and tested using both legitimate and phishing QR code datasets. Different evaluation metrics such as accuracy, precision, recall, F1-score, confusion matrix, and training-validation graphs were used to compare the performance of the models. Among all the models, EfficientNetB0, MobileNetV2, and CNN achieved the highest accuracy of around 99%. The overall results show that deep learning-based image analysis can play an important role in improving QR code security and phishing detection systems.

5.2 Limitations

Despite achieving high accuracy, the research had some limitations. Certain models required a large amount of computational power and longer training time. In some cases, visually similar legitimate and phishing QR codes created difficulties during classification. The research mainly focused on QR image analysis and did not include real-time webpage or URL behavior analysis after scanning the QR code. Limited hardware resources and dataset quality also affected the overall training and testing process.

5.3 Future Improvement

In the future, this research can be improved by using larger and more diverse real-world QR phishing datasets. Real-time QR code scanning systems for mobile and web platforms can also be developed. Additional security features such as malicious URL analysis, fake website detection, and logo similarity detection may further improve the performance of the system. More advanced and lightweight deep learning models can also be explored to make the detection process faster, smarter, and more suitable for real-time cybersecurity applications.

References

1. S. K. Fayaz, M. Rahman, and T. Ahmed, "QR Code Security: Mitigating the Issue of Quishing," *International Journal of Cyber Security*, vol. 12, no. 3, pp. 45–53, 2023.
2. A. Kumar and R. Singh, "QRishing: The Susceptibility of Smartphone Users to QR Code Phishing Attacks," *Proceedings of the International Conference on Cyber Security*, pp. 112–118, 2022.
3. A. M. Alshamrani and M. A. Alsubhi, "Prevention of Phishing Attacks Using QR Code Safe Authentication," Springer Nature, 2021.
4. Y. Kaya and H. Pehlivan, "Cyber Attack Detection with QR Code Images Using Lightweight Deep Learning Models," *ScienceDirect*, 2022.
5. S. Schrittwieser and M. Mulazzani, "Malicious Pixels Using QR Codes as Attack Vector," Springer Nature, 2011.
6. M. R. Islam and S. Akter, "Secure QR Code Scanner to Detect Malicious URL using Machine Learning," *IEEE Conference Publication*, 2022.
7. M. Abadi et al., "TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems," 2015. [Online]. Available: <https://www.tensorflow.org>
8. F. Chollet et al., "Keras," 2015. [Online]. Available: <https://keras.io>
9. Python Software Foundation, "Python Programming Language," [Online]. Available: <https://www.python.org>
10. OpenCV Library, "Open Source Computer Vision Library," [Online]. Available: <https://opencv.org>
11. J. Deng et al., "ImageNet: A Large-Scale Hierarchical Image Database," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2009, pp. 248–255.
12. Scikit-learn Developers, "Scikit-learn: Machine Learning in Python," [Online]. Available: <https://scikit-learn.org>
13. Dataset-https://www.kaggle.com/datasets/kaggleprollc/phishing-url-websites-dataset-phiusiil?resource=download&select=PhiUSIIL_Phishing_URL_Dataset.csv